



**R. C. PATEL
INSTITUTE OF TECHNOLOGY**
An Autonomous Institute

Laboratory Manual

Microcontroller and Applications

[RCP23ELPC253L]

**Department of Electronics & Telecommunication
Engineering**



R C Patel Institute of Technology, Shirpur

AY: 2025-26

Vision of the Institute

Fostering technical excellence through ethics, sustainable education, innovation and research.

Mission of the Institute

To impart high quality Technical Education through:

M1. Innovative and Interactive learning process and high quality, globally recognized instructional programs.

M2. Fostering a collaborative scientific temper among students with ethical responsibility towards the society.

M3. Preparing students from diverse backgrounds to have aptitude for employment, entrepreneurship and research with a spirit of Professionalism.

M4. To contribute to nation's sustainable development.

Vision of the Department

To produce competent Electronics and Telecommunication Engineers with a commitment to serve society.

Mission of the Department

M1: Deliver innovative, ICT-enabled, and interactive learning, fostering excellence in electronics and telecommunication engineering.

M2: Cultivate scientific inquiry, ethical responsibility, and collaboration to address societal and technological challenges effectively.

M3: Empower diverse students with skills for employment, entrepreneurship, and research, nurturing professionalism and lifelong learning.

Program Educational Objectives (PEOs) of the Department

PEO1 Graduates will apply engineering knowledge and innovation to solve complex problems in Electronics and Telecommunication domains.

PEO2 Graduates will demonstrate ethical and social values while adapting to technological advancements for sustainable societal progress.

PEO3 Graduates will pursue careers through employment, entrepreneurship, or research, embracing professionalism and continuous learning throughout life.

Program Specific Outcomes (PSOs)

PSO1 Communication Systems: Design and analyze analog and digital communication systems for wireless, optical, and network-based applications.

PSO2 Signal Processing and Embedded Systems: Apply signal processing and embedded system design using modern tools to develop intelligent systems, automation, and smart electronic products.

Program Outcomes:

Engineering graduates will be able to:

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the Engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Microprocessor and Applications Laboratory

[RCP23ELPE601]

Pre-requisite

1. Basic Electrical Engineering & Digital Electronics
2. Digital System Design

Course Objectives

1. To develop background knowledge and core expertise in microcontrollers.
2. To understand peripheral devices and their interfacing to microcontrollers.
3. To write programs for microcontrollers and their applications in Assembly language.

COs	Course Objectives	Blooms Level	Blooms Description
CO1	Identify the different components of Microcomputer.	L1	Comprehension
CO2	Identify the functionality of different blocks of 8051 Microcontroller.	L2	Understand
CO3	Write assembly language programs for microcontroller based systems using instruction set.	L3	Apply
CO4	Interface different input/output devices with microcontroller for various applications	L4	Analyze



The Shirpur Education Society's

R. C. Patel Institute of Technology, Shirpur

CERTIFICATE

This is to certify that Mr. / Miss _____
of **S.Y.B.Tech Branch: Electronics & Telecommunication**
PRN:_____ has performed practical work
satisfactorily in the **Course: Microcontroller and Applications**
Laboratory [RCP23EPC253L] in the premises of the Department of
Electronics & Telecommunication Engineering during the academic
year 2025-26.

Date:

Signature of the Course Coordinator

Place: Shirpur

Office Seal

Head of Department

LIST OF EXPERIMENTS

Expt. No.	Name of the Experiment	CO Mapped	Performed Date	Page No	Faculty Signature
1	Study of the AVR microcontroller development board	CO1			
2	Programs on arithmetic instructions	CO3			
3	Programs on Data Transfer instructions	CO3			
4	Program on flag bits in status register and related instructions	CO3			
5	Interfacing display – LED	CO3			
6	Interfacing a 7 Segment Display	CO3			
7	Ambient temperature display (using typically DHT11 or a suitable sensor)	CO3			
8	Interfacing Servo Motor (dc-continuous/stepper/servo etc).	CO3			



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 1: Identify the different components of Microcomputer	
CO 2: Identify the functionality of different blocks of 8051 Microcontroller.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 01

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Divison:		Roll No:	
Performed Date:		Checked Date	

Aim:**Study of AVR Microcontroller Board****Objectives:**

- To understand the architecture and features of the AVR microcontroller.
- To study the pin configuration and functions of the AVR development board.
- To learn the basic interfacing of input/output devices with AVR.
- To familiarize with programming and execution using AVR software tools

Pre-Lab Questions:

- What is an AVR microcontroller? Explain its basic architecture.
- List the main features of AVR microcontrollers.
- Explain the function of I/O ports (GPIO) in AVR microcontroller.
- What is the role of clock and reset circuit in AVR board?
- Differentiate between microcontroller and microprocessor.

Equipment Required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	AVR Microcontroller Trainer/Development Board	ATmega16/ATmega32 based	1
2	USB Programmer (USBasp)	ISP Programming Support	1
3	Power Supply	+5V DC Regulated	1
4	Computer System	AVR IDE installed (Atmel Studio/Arduino IDE)	1

Theory:

ATmega32 – 8 Bit AVR Microcontroller- AVR microcontrollers utilize the advanced RISC architecture, with the ATmega32 being an 8-bit microcontroller designed on the AVR enhanced RISC architecture and characterized by low power consumption. In the context of a 1MHz cycle frequency, AVR microcontrollers can execute one million instructions per second.



Figure 1. IC of ATMEGA 39p

Key Features:

- 32 general-purpose registers arranged in an 8×32 configuration.
- 32 kilobytes of flash program memory that is in-system self-programmable.
- 2 kilobytes of internal SRAM (Static Random-Access Memory).
- 1024 bytes of EEPROM (Electrically Erasable Programmable Read-Only Memory).
- Available in 40-pin DIP (Dual Inline Package), 44-lead QTFP (Quad Flat Thin Package), and 44-pad QFN/MLF (Quad Flat No-Lead/Micro Lead Frame).
- 32 programmable I/O (Input/Output) lines for versatile connectivity.
- 8-channel, 10-bit ADC (Analog-to-Digital Converter) for precise analog signal conversion.
- Two 8-bit timers/counters featuring separate prescalers and compare modes.
- One 16-bit timer/counter with a separate prescaler, compare mode, and capture mode.

- 4 PWM (Pulse Width Modulation) channels for effective motor and light control.
- In-system programming facilitated by an on-chip boot program.
- Programmable watchdog timer with a separate on-chip oscillator for enhanced system reliability.
- Programmable serial USART (Universal Synchronous/Asynchronous Receiver/Transmitter) for serial communication.
- Master/slave SPI (Serial Peripheral Interface) serial interface for efficient data exchange.

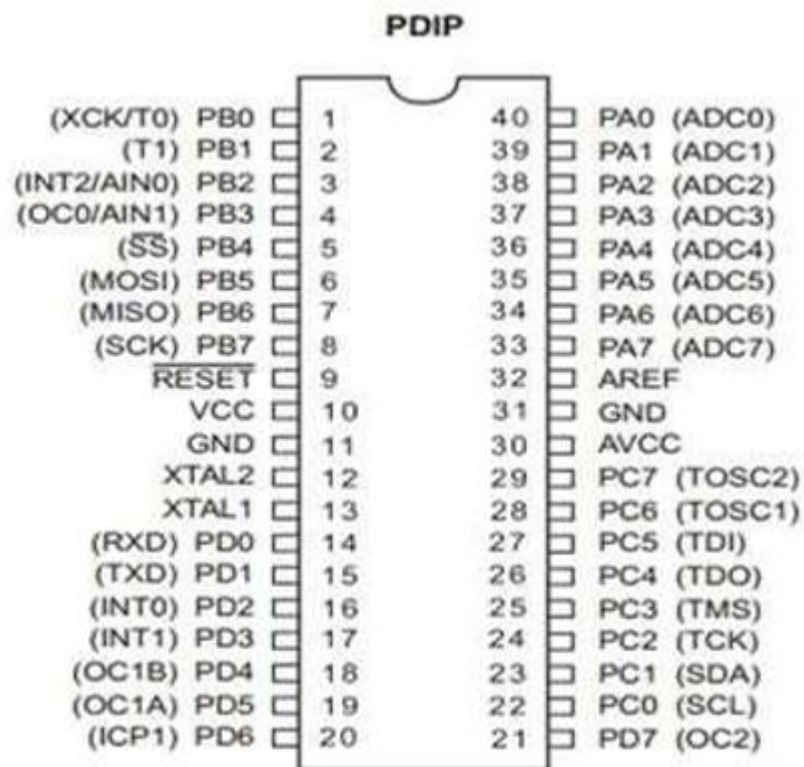


Figure 2. Pin Diagram of ATMEGA39P

Special Microcontroller Features:

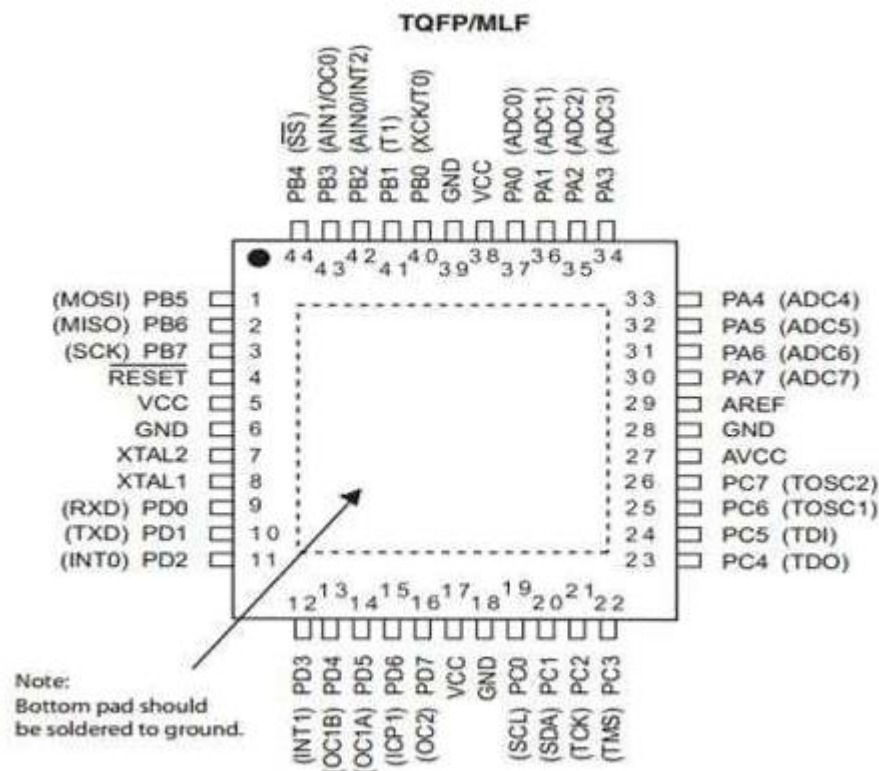
- Six sleep modes available: Idle, ADC noise reduction, power-save, power-down, standby, and extended standby.
- Utilizes an internal calibrated RC oscillator for precise timing.
- Supports both external and internal interrupt sources for flexible event triggering.
- Incorporates power-on reset functionality and programmable brown-out detection for enhanced system reliability.

All 32 registers are directly linked to the Arithmetic Logic Unit (ALU), enabling the access of

two independent registers in a single instruction executed within a single clock cycle. During power down, the register contents are preserved, but the oscillator is halted, resulting in the deactivation of all other chip functions until an external interrupt occurs. The asynchronous timer permits the maintenance of a timer-based power-save mode while the remainder of the device remains in a sleep state.

In ADC noise reduction mode, the CPU and all I/O modules, except ADC and the asynchronous timer, are halted. In standby mode, the entire device, with the exception of the crystal oscillator, enters a sleep state. Extended standby mode ensures that both the main oscillator and asynchronous timer continue to operate.

The ATmega32 stands out as a robust microcontroller due to its in-system self-programmable flash integrated on a monolithic chip. This feature provides a highly flexible and cost-effective solution for a variety of embedded control applications.



Pin Descriptions:

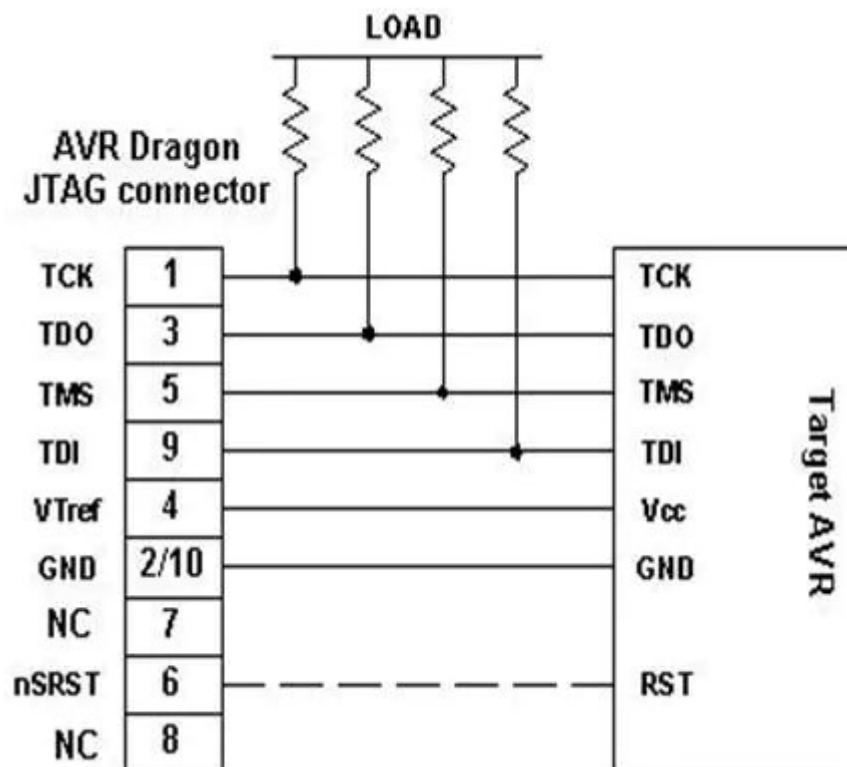
VCC: Supply of digital voltage

GND: Reference to the ground

Port A (PA7-PA0): This port functions as analog inputs for the A/D converter and operates as an 8-bit bidirectional I/O port in the absence of A/D converter usage.

Port B (PB7-PB0) & Port D (PD7-PD0): These are 8-bit bidirectional I/O ports with symmetrical drive characteristics in their output buffers, offering both high sink and source capabilities. When configured as inputs, they are significantly pulled low when pull-up resistors are engaged. Additionally, these ports serve various special function features of the ATmega32. Port C (PC7-PC0): An 8-bit bidirectional I/O port where, if the JTAG interface is enabled, the pull-up resistors on pins PC5 (TDI), PC3 (TMS), and PC2 (TCK) will be activated.

Port C (PC7-PC0): An 8-bit bidirectional I/O port where, if the JTAG interface is enabled, the pull-up resistors on pins PC5 (TDI), PC3 (TMS), and PC2 (TCK) will be activated.



Reset: – An input signal that initiates a system reset.

XTAL1: – Serves as both an input to the inverting oscillator amplifier and as an input to the internal clock operating circuit.

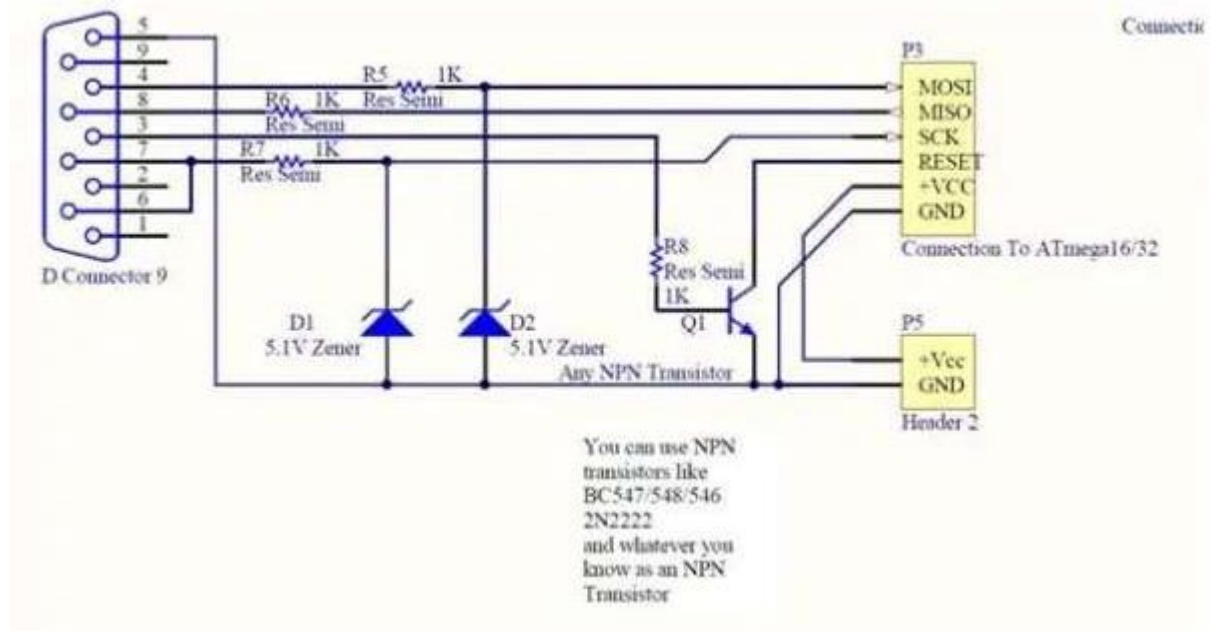
XTAL2: – Outputs from the inverting oscillator amplifier, contributing to the external clock signal.

AVCC: – The supply voltage pin specifically designated for Port A and the A/D converter. It must be connected to VCC.

AREF: – AREF functions as the analog reference pin for the A/D converter.

ATmega32 Memories: – This microcontroller features two primary memory spaces: data memory and program memory space. Additionally, it includes EEPROM memory for persistent data storage.

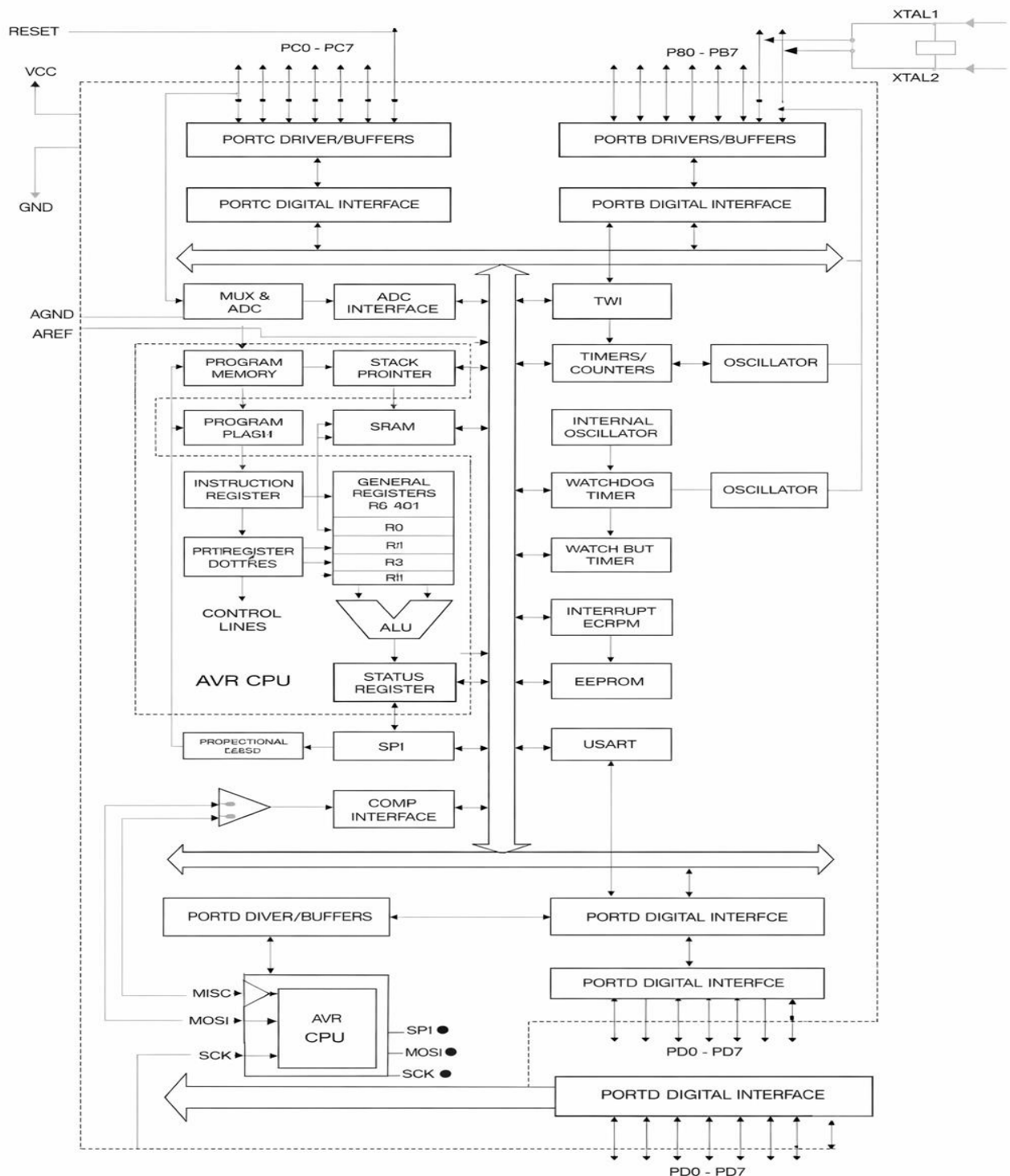
In-System Programmable Flash Program Memory: – ATmega32 incorporates 32 kilobytes of on chip in-system reprogrammable flash memory for program storage. The flash memory is organized as 16k x 16 and is divided into two sections: the Boot program section and the application program section.



SRAM Data Memory: The lower 2144 data memory locations encompass the addressing of the Register file, I/O memory, and internal data SRAM. The initial 96 locations target the Register file and I/O memory, while the subsequent 2048 locations are dedicated to addressing the internal data SRAM. There are five distinct addressing modes for data memory coverage: direct, indirect with displacement, indirect, indirect with pre-decrement, and indirect with post-decrement. Utilizing these addressing modes allows access to the 32 general-purpose registers, 64 I/O registers, and 2048 bytes of internal data SRAM.

EEPROM Data Memory: It encompasses a data EEPROM memory of 1024 bytes, accessible as a distinct data space for individual byte reading and writing. **I/O Memory:** All input/output operations and peripherals are situated within the I/O space.

The IN and OUT instructions are employed to transfer data between the 32 general-purpose registers and the I/O space. I/O registers within the address range 00-1F are directly manipulated at the bit level using the SBI and CBI instructions. **Introduction to ATmega8:** The ATmega8 belongs to the AVR family, a 8-bit CMOS microcontroller developed by Atmel Corporation in 1996. It operates on the RSIC (Reduced Instruction Set Computer) architecture. One of its key advantages lies in the absence of an accumulator, allowing the result of any operation to be stored in a register defined by the instruction.

Architecture:**Figure: Internal Architecture of AVR**

The above diagram represents the internal block architecture of an **AVR microcontroller**. At the center is the AVR CPU consisting of the ALU, general purpose registers, instruction decoder, program counter, stack pointer and status register which together execute instructions. The controller follows Harvard architecture with separate **Program Flash memory** and

SRAM data memory. Surrounding the CPU are peripheral modules such as timers/counters, interrupt unit, watchdog timer, EEPROM and oscillator circuits that control timing and system operation. The microcontroller also includes communication interfaces like **USART, SPI and TWI (I²C)** for serial data transfer. Analog functionality is provided through the **ADC and analog comparator**, while multiple digital I/O ports (PORTA, PORTB, PORTC, PORTD) with drivers and buffers allow connection to external devices. All blocks communicate through an internal data bus, enabling the AVR to perform computation, control and interfacing in embedded applications.

Memory- It is equipped with 8 kilobytes of flash memory, 1 kilobyte of SRAM, and 512 bytes of EEPROM. The 8K flash is divided into two sections – the lower part functions as the boot flash section, while the upper part serves as the application flash section. Within the SRAM, there are 1 kilobyte along with 1120 bytes allocated for general-purpose registers and I/O registers. The lower 32 address locations are dedicated to 32 general-purpose 8-bit registers, while the subsequent 64 addresses are assigned to I/O registers, all directly linked to the Arithmetic Logic Unit (ALU). The EEPROM is utilized for storing user-defined data.

In terms of input/output ports, the configuration includes 23 I/O lines distributed across three ports, namely B, C, and D. Port B encompasses 8 I/O lines, Port C comprises 7 I/O lines, and Port D consists of 8 I/O lines. The registers corresponding to any port X (B, C, or D) are as follows: DDRX (Port X data direction register), PORTX (Port X data register), and PINX (Port X input register).

The device also features three timers with comparable modes, including two 8-bit timers and one 16-bit timer.

Oscillators: It includes an internal reset and oscillator, eliminating the requirement for external inputs. The internal RC oscillator can generate a clock with programmable frequencies of 1MHz, 2MHz, 4MHz, or 8MHz. Additionally, it accommodates an external oscillator with a maximum frequency of 16MHz.

Communication: It offers both synchronous and asynchronous data transfer options through the Universal Synchronous and Asynchronous Receiver Transmitter (USART), enabling communication with modems and various serial devices. Additionally, it supports the Serial Peripheral Interface (SPI) for communication between devices using the master-slave method. Another supported communication type is the Two-Wire Interface (TWI), facilitating communication between two devices through 2 wires and a common ground connection. The chip integrates a comparator module that enables voltage comparison between two inputs connected to external chips. Furthermore, it features a 6-channel Analog-to-Digital Converter (ADC), with 4 channels providing 10-bit accuracy and 2 channels offering 8-bit accuracy.

Status Register: This register contains information about the currently executed arithmetic instruction set.

Microcontroller Sleep Modes

The microcontroller features six sleep modes, each serving distinct purposes.

1. Idle Mode: – This mode halts the CPU's operation while allowing the SPI, USART, ADC, TWI, Timer/Counter, and Watchdog, as well as the interrupt system, to function.
Activation involves setting the SM0 to SM2 bits of the MCU register flag to zero.
2. ADC Noise Reduction Mode: – This mode stops the CPU while permitting the operation of the ADC, external interrupts, Timer/Counter2, and the watchdog.
3. Power Down Mode: – External interrupts, the 2-wire serial interface, and the watchdog are enabled in this mode, while the external oscillator is disabled, leading to the cessation of all generated clocks.
4. Power save Mode: – Employed when the Timer/Counter is clocked asynchronously, this mode halts all clocks except clk ASY.
5. Standby Mode: – In Standby mode, the oscillator continues to operate, while all other operations come to a halt.

Conclusion:

Post Lab questions:

1. What are the functions of **DDRx**, **PORTx** and **PINx** registers in AVR?
2. Why is a current-limiting resistor required when interfacing an LED?
3. What is the role of the clock source in AVR microcontroller operation?
4. How can you change the LED blinking speed in the program?
5. What will happen if the reset pin is activated during program execution?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 02

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: To understand and implement arithmetic instructions using AVR Assembly Language.

- I. **LDI — Load Immediate**
- II. **ADD — Add without Carry**
- III. **SUB — Subtract without Borrow**
- IV. **INC — Increment**
- V. **DEC — Decrement**

Objectives

- To understand arithmetic instructions of AVR assembly language (ADD, ADC, SUB, INC, DEC, MUL).
- To perform addition and subtraction of 8-bit numbers using AVR assembly program.
- To understand the effect of arithmetic operations on status flags (Zero, Carry, Sign, and overflow).
- To execute and verify the output using AVR simulator/trainer board.

Pre-Lab Questions

1. What is the difference between **ADD** and **ADC** instructions in AVR?
2. Define the function of Status Register (SREG) in AVR microcontroller.
3. What happens to the Carry flag after addition of two 8-bit numbers?
4. Write the general format of an AVR assembly instruction.
5. What is the purpose of **INC** and **DEC** instructions?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	AVR Microcontroller Trainer/Development Board	ATmega16/ATmega32 based	1
2	USB Programmer (USBasp)	ISP Programming Support	1
3	Power Supply	+5V DC Regulated	1
4	Computer System	AVR IDE installed (Atmel Studio/Arduino IDE)	1

Theory:

AVR microcontrollers are 8-bit RISC processors that perform arithmetic operations directly on register data. The arithmetic instructions operate on the general purpose registers (R0–R31) and the result is stored back into a register. During execution, the Status Register (SREG) is automatically updated to indicate the result of the operation.

The SREG contains important flags:

- **C (Carry)** – set when result exceeds 8 bits
- **Z (Zero)** – set when result = 0
- **N (Negative)** – set when MSB = 1
- **V (Overflow)** – signed overflow
- **S (Sign)** – $N \oplus V$

These flags help in decision making and program control.

Programs and Explanations:

1. Addition of 19 and 25

LDI r16, 19 ; Load 19 into register r16

LDI r17, 25 ; Load 25 into register r17

ADD r16, r17 ; Add r17 to r16, result in r16

Result: $r16 = 19 + 25 = 44$ (0x2C)

2. Addition of \$19, \$25 and \$5

LDI r16, 0x19 ; Load \$19 into r16

LDI r17, 0x25 ; Load \$25 into r17

ADD r16, r17 ; $r16 = \$19 + \25

LDI r18, 0x05 ; Load \$5 into r18

ADD r16, r18 ; $r16 = r16 + \$5$

Result: $r16 = 0x19 + 0x25 + 0x05 = 0x43$

3. Subtraction of Two Numbers

LDI r16, 50 ; Load 50 into r16

LDI r17, 20 ; Load 20 into r17

SUB r16, r17 ; Subtract r17 from r16

Result: r16 = 50 - 20 = 30 (0x1E)

4. Load Number in Different Formats and Increment/Decrement

LDI r16, 0x1A ; Load Hexadecimal 1A into r16

INC r16 ; Increment r16 by 1 -> 0x1B

LDI r17, 26 ; Load Decimal 26 into r17

DEC r17 ; Decrement r17 by 1 -> 25

LDI r18, 0b11001100 ; Load Binary 11001100 into r18

INC r18 ; Increment r18 by 1 -> 0b11001101

Results: - r16 = 0x1B - r17 = 25 - r18 = 0b11001101

Results:

- r16 = 0x1B

- r17 = 25

- r18 = 0b11001101

Conclusion:

Post-Lab Questions

1. Which flags are affected after subtraction operation in AVR?
2. Why is **ADC** used in multi-byte addition?
3. What will happen if the result of an arithmetic operation exceeds 8 bits?
4. How can you verify the output of the program in the simulator?
5. Differentiate between **SUB** and **SBC** instructions.



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 03

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: To implement instructions involving immediate, register, and internal data memory operands using AVR Assembly Language.

Instructions Covered:

- **LDS — Load Direct from Data Space**
- **STS — Store Direct to Data Space**
- **ADD — Add without Carry**
- **SUB — Subtract without Borrow**
- **MUL — Multiply**
- **AND — Logical AND**
- **INC — Increment**
- **DEC — Decrement**

Objectives

- To understand different operand addressing types in AVR assembly language.
- To implement instructions using **immediate operands**.
- To perform operations using **register operands**.

Pre-Lab Questions

1. What is meant by an operand in assembly language?
2. What is the difference between immediate and register addressing mode?
3. Which instruction is used to load data from SRAM into a register?
4. What is the purpose of pointer registers in AVR?
5. What is internal data memory in AVR microcontroller?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	AVR Microcontroller Trainer/Development Board	ATmega16/ATmega32 based	1
2	USB Programmer (USBasp)	ISP Programming Support	1
3	Power Supply	+5V DC Regulated	1
4	Computer System	AVR IDE installed (Atmel Studio/Arduino IDE)	1

Theory:

In AVR microcontrollers, instructions operate on data stored in registers and internal data memory (SRAM). Depending on where the operand is located, the instruction may use register operands or memory operands. Arithmetic and logical operations are generally performed in registers, while special instructions are used to transfer data between registers and SRAM.

Tasks and Codes:**1. Load \$30 and \$97 into \$105 and \$106**

```
LDI r16, 0x30
```

```
STS 0x0105, r16
```

```
LDI r16, 0x97
```

```
STS 0x0106, r16
```

2. Load \$55 into \$300–\$308

```
LDI r16, 0x55
```

```
STS 0x0300, r16
```

```
STS 0x0301, r16
```

```
STS 0x0302, r16
```

```
STS 0x0303, r16
```

```
STS 0x0304, r16
```

```
STS 0x0305, r16
```

STS 0x0306, r16

STS 0x0307, r16

STS 0x0308, r16

3. Load \$5F into PORTB

LDI r16, 0x5F

OUT PORTB, r16

4. (a) Load \$11 into \$100–\$105, (b) Add values into R20 as they are added

LDI r16, 0x11

STS 0x0100, r16

STS 0x0101, r16

STS 0x0102, r16

STS 0x0103, r16

STS 0x0104, r16

STS 0x0105, r16

LDI r20, 0x00

LDS r16, 0x0100

ADD r20, r16

LDS r16, 0x0101

ADD r20, r16

LDS r16, 0x0102

ADD r20, r16

LDS r16, 0x0103

ADD r20, r16

LDS r16, 0x0104

ADD r20, r16

LDS r16, 0x0105

ADD r20, r16

4. (Repeat) Place result in \$105 after addition

STS 0x0105, r20

5. (a) Load \$15 into \$67, (b) Add to R19 five times

LDI r16, 0x15

STS 0x0067, r16

LDI r19, 0x00

LDS r16, 0x0067

ADD r19, r16

ADD r19, r16

ADD r19, r16

ADD r19, r16

ADD r19, r16

5. (Repeat) Place result in \$67

STS 0x0067, r19

6. Complement contents of \$68 and place in R27

LDS r16, 0x0068

COM r16

MOV r27, r16

7. Copy data from \$68 to PORTC using R19

LDS r19, 0x0068

OUT PORTC, r19

Result:

Conclusion:**Post-Lab Questions**

1. How is data stored into SRAM using AVR assembly instructions?
2. What is the difference between **LDS/STS** and **LD/ST** instructions?
3. Why are pointer registers required for memory access?
4. Which addressing mode executes faster and why?
5. How can you verify data transfer between register and memory?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 04

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: To understand the AVR status register, its flag bits, and observe their behaviour with arithmetic and logical instructions.

Objectives

- I. To study the structure of the AVR Status Register (SREG).
- II. To understand the function of each flag bit (C, Z, N, V, S, H, T, I).
- III. To observe how arithmetic and logical instructions affect these flags.
- IV. To analyze program decisions based on flag conditions.

Pre-Lab Questions

1. What is the purpose of the Status Register (SREG) in AVR microcontroller?
2. List all the flag bits present in SREG.
3. Which flag indicates that the result of an operation is zero?
4. What is the difference between Carry flag and Overflow flag?
5. Which instructions commonly modify the status flags?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	AVR Microcontroller Trainer/Development Board	ATmega16/ATmega32 based	1
2	USB Programmer (USBasp)	ISP Programming Support	1
3	Power Supply	+5V DC Regulated	1
4	Computer System	AVR IDE installed (Atmel Studio/Arduino IDE)	1

In AVR microcontrollers, every arithmetic or logical operation affects certain condition flags stored in a special register called the Status Register (SREG). The SREG helps the CPU make decisions during program execution, especially in conditional branching, comparisons and loops. The SREG is an 8-bit register and each bit represents a flag that indicates the result of the last executed instruction.

Bit	Flag	Name	Function
7	I	Global Interrupt Enable	Enables/disables interrupts
6	T	Bit Copy Storage	Used in bit copy instructions
5	H	Half Carry	Carry from bit3 → bit4 (BCD operations)
4	S	Sign Flag	$S = N \oplus V$
3	V	Overflow	Signed overflow occurred
2	N	Negative	MSB of result is 1
1	Z	Zero	Result equals zero
0	C	Carry	Carry/borrow occurred

Programs and Explanations:

1. Complement contents of \$68 and place in R27

LDS r16, 0x0068

COM r16

MOV r27, r16

2. Copy data from \$68 to PORTC using R19

LDS r19, 0x0068

OUT PORTC, r19

3. Status Register Details

- The status register is an 8-bit register.

4. C and H Flag Bits

- The C (Carry) flag is bit 0.
- The H (Half Carry) flag is bit 5.

5. V and N Flag Bits

- The V (Two's Complement Overflow) flag is bit 3.
- The N (Negative) flag is bit 2.

6. H Flag Behavior in ADD Instruction

- In the ADD instruction, the H flag is raised if there is a carry from bit 3 to bit 4.

7. Status of C and Z Flags after Following Code

LDI R20, 0xFF

LDI R21, 0x01

ADD R20, R21

- $C = 1$ (Carry out from bit 7)
- $Z = 1$ (Result is 0)

8. C Flag Value in Different Cases

(a)

LDI R20, 0x54

LDI R25, 0xC4

ADD R20, R25

- $C = 1$ ($0x54 + 0xC4 = 0x118 \rightarrow$ Carry out)

(b)

LDI R23, 0x00

LDI R16, 0xFF

ADD R23, R16

- $C = 0$ ($0x00 + 0xFF = 0xFF \rightarrow$ No carry)

(c)

LDI R30, 0xFF

LDI R18, 0x05

ADD R30, R18

- $C = 1$ ($0xFF + 0x05 = 0x104 \rightarrow$ Carry out)

9. Add 0x55 Five Times

LDI r16, 0x55

LDI r20, 0x00

ADD r20, r16

ADD r20, r16

ADD r20, r16

ADD r20, r16

ADD r20, r16

Result:

Conclusion:

Post-Lab Questions

1. What happens to the Zero flag after an operation producing zero result?
2. How does the Carry flag behave after addition and subtraction?
3. Why is the Overflow flag important in signed arithmetic operations?
4. Which flag indicates negative result in AVR?
5. How can program execution be controlled using status flags?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 05

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: Interfacing of LED with Arduino.

Objectives:

- To understand digital output pins of Arduino
- To learn basic interfacing of LED
- To write and upload Arduino program
- To generate blinking output

Pre-Lab Questions – Interfacing of LED with Arduino

1. What is a digital output pin in Arduino?
2. Why is a current-limiting resistor required while connecting an LED?
3. What is the function of `pinMode()` in Arduino programming?
4. What do HIGH and LOW signals represent in Arduino?
5. What is the purpose of the `delay()` function?

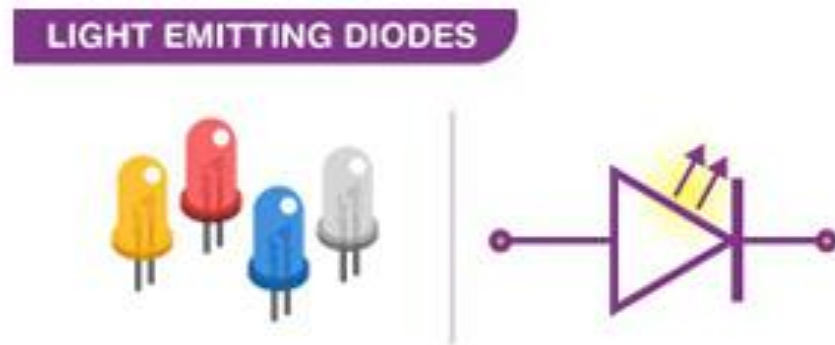
Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	Arduino Board	Arduino Board	1
2	Breadboard	-	1
3	Power Supply	+5V DC Regulated	1
4	LED	1.5V	1
5	220 ohm resistor	220 Ohm resistor	1

Theory:

What is LED?

A light-emitting diode (LED) is a semiconductor device that emits light when an electric current flows through it. When current passes through an LED, the electrons recombine with holes emitting light in the process. LEDs allow the current to flow in the forward direction and blocks the current in the reverse direction. Light-emitting diodes are heavily doped p-n junctions. Based on the semiconductor material used and the amount of doping, an LED will emit colored light at a particular spectral wavelength when forward biased. As shown in the figure, an LED is encapsulated with a transparent cover so that emitted light can come out.



Arduino Board:

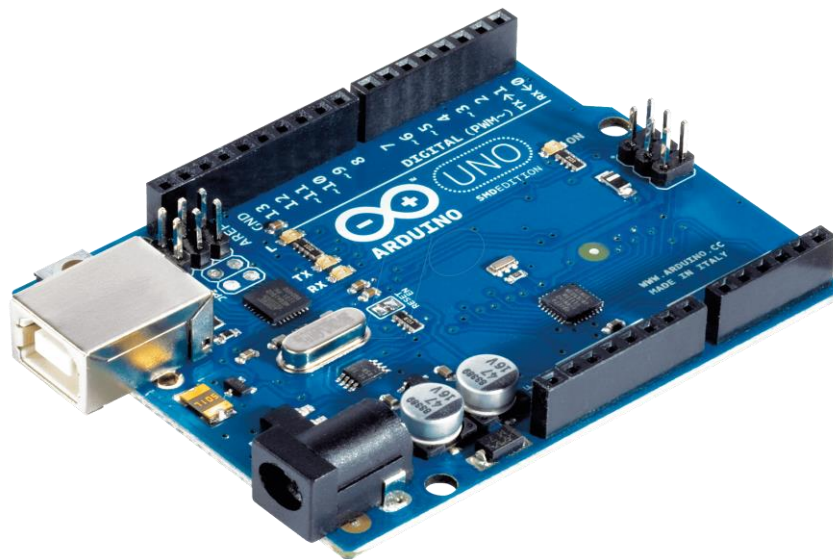


Fig. Arduino-Uno Board

Arduino Uno is an open-source microcontroller development board based on the ATmega328P microcontroller. It is widely used in education and prototyping because of its simple hardware and easy programming through the Arduino IDE. The board can read inputs from sensors, switches or other devices and control outputs such as LEDs, motors and displays.

The Arduino Uno operates at 5V and contains 14 digital input/output pins, out of which 6 pins support PWM, and 6 analog input pins for analog signal measurement. It has a 16 MHz crystal oscillator,

USB interface for programming and power supply, power jack, reset button and ICSP header. Programs (called sketches) are written in a simplified C/C++ language and uploaded directly to the board through a USB cable.

The board also includes built-in communication interfaces such as UART, SPI and I2C, making it suitable for interfacing with sensors, modules and other microcontrollers. Due to its reliability, low cost and extensive library support, Arduino Uno is commonly used in embedded systems experiments, automation projects and IoT applications.

Pin Description of Arduino Uno Board

The Arduino Uno board consists of several types of pins used for power supply, input/output operations and communication with external devices.

1. Digital Input/Output Pins (0–13)

- Total **14 digital pins**
- Can be configured as **INPUT or OUTPUT** using `pinMode()`
- Operate at **5V logic level**

Special Pins:

- **0 (RX) and 1 (TX)** → Serial communication (UART)
- **~3, 5, 6, 9, 10, 11** → PWM pins (used for analog output using `analogWrite()`)
- **10, 11, 12, 13** → SPI communication pins
- **13** → Connected to on board LED

Source Code

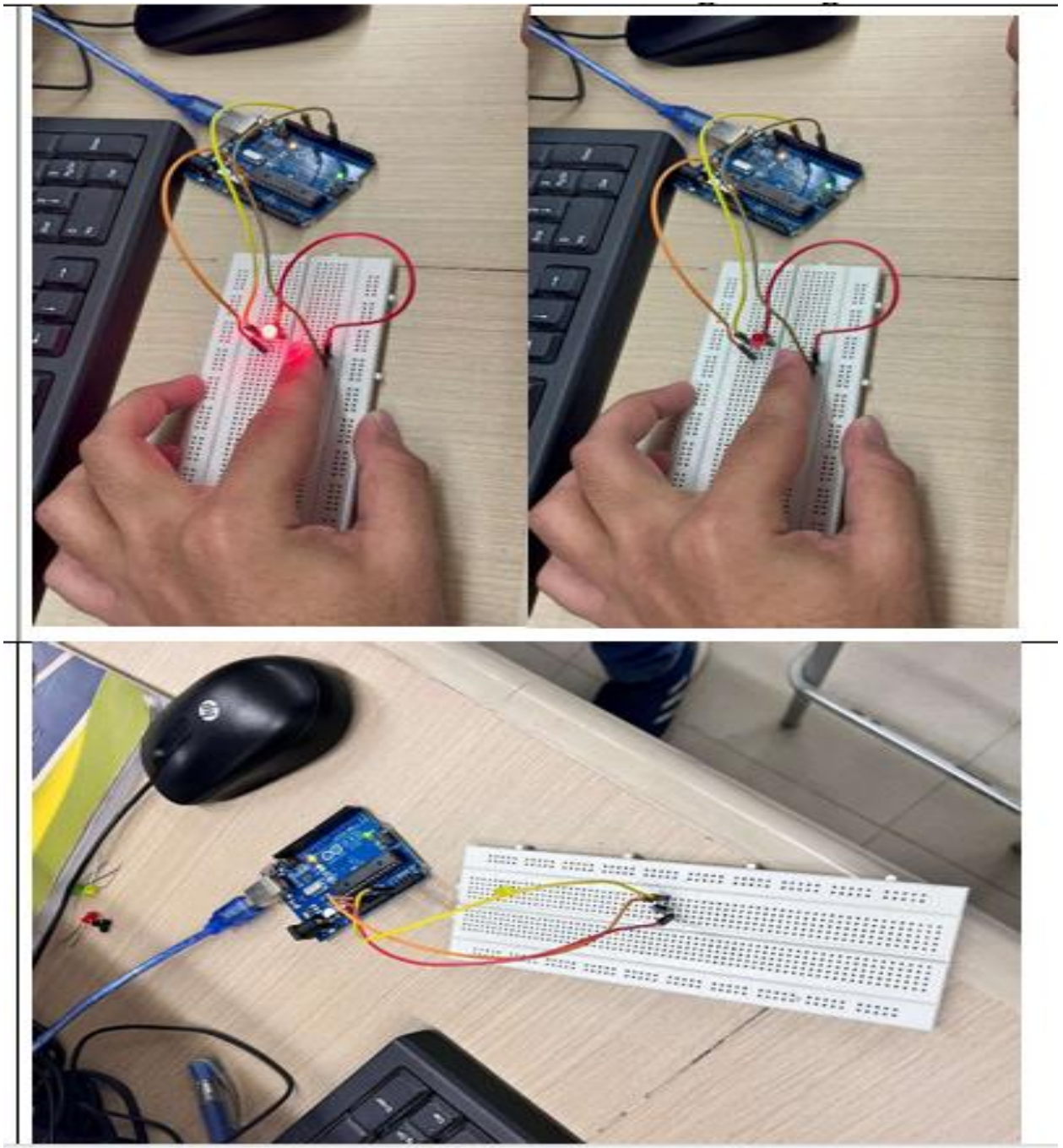
```
/*  
  Blink  
  Turns on an LED on for one second, then off for one second, repeatedly.  
*/  
  
// the setup function runs once when you press reset or power the board  
  
void setup() { // initialize digital pin 13 as an output.  
  pinMode(2, OUTPUT);  
}  
  
// the loop function runs over and over again forever  
  
void loop() {  
  digitalWrite(2, HIGH); // turn the LED on (HIGH is the voltage level)  
  delay(1000); // wait for a second  
  digitalWrite(2, LOW); // turn the LED off by making the voltage LOW  
  delay(1000); // wait for a second  
}
```

Blinking LED specific number of times

```
void blink3( )  
  
{  
  
for (int count = 0; count < 3; count++)  
  
{  
  
digitalWrite(led1, HIGH);  
  
delay(500);  
  
digitalWrite(led1, LOW);  
  
delay(500);  
  
}  
  
}
```

Blinking Alternate LED**Source Code**

```
/*      Alternate Blink  
        1. Turn on LED1, turn off LED2 for 1 second (at the same time)  
        2. Turn off LED1, turn on LED2 for 1 second (at the same time)  
        3. Repeat  
*/  
  
void setup() {  
  pinMode(9, OUTPUT);  
  pinMode(8, OUTPUT);  
  
}  
  
void loop() {  
  //turn on pinA and turn off pinB  
  digitalWrite(9, HIGH);  
  digitalWrite(8, LOW);  
  
  delay(1000);    //wait 1 sec  
  //turn off pinA and turn on pinB  
  digitalWrite(9, LOW);  
  digitalWrite(8, HIGH);  
  
  delay(1000);   //wait 1 sec  
}
```



Result:

Conclusion:

Post-Lab Questions

1. Why resistor is required with LED?
2. What is digital output pin?
3. What happens if HIGH is replaced by LOW?
4. How to change blinking speed?
5. What is the function of `pinMode()`?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 06

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: Interfacing with Seven Segment Display

Objectives:

- I. To understand the working of seven segment display
- II. To learn digital output control using Arduino
- III. To display numeric characters on display
- IV. To understand segment mapping logic

Pre-Lab Questions – Interfacing with Seven Segment Display:

1. What is a seven segment display and how does it work?
2. What is the difference between **common anode** and **common cathode** display?
3. Why are current limiting resistors required for each segment?
4. How many segments are present in a seven segment display and how are they labelled?
5. Which logic level turns ON a segment in a common cathode display?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	Arduino Board	Arduino Board	1
2	Breadboard	-	1
3	Power Supply	+5V DC Regulated	1
4	7 Segment Display	1.5V	1
5	220 ohm resistor	220 Ohm resistor	1

Theory:

7-segment displays are made up of 8 LED segments as shown in fig 9.1 below. They are used to display Numbers (0-9) and certain Alphabets (like A, B, Cs etc.).

- 7 of these LED segments are in the shape of a line, whereas 1 segment is circular.
- Each of the 8 elements has a pin associated with it which can be driven HIGH or LOW.
- To display a number or alphabet, we need to turn on specific LED segments of the display.

There are two types of seven-segment displays: common anode and common cathode as shown in fig 9.2 below. In common cathode seven-segment displays, all seven LEDs, in addition to a dot LED, have the cathodes connected to pins 8 and pin 3. To use this display, we must connect GROUND to pin 3 and pin 8, then connect +5V to the other pins and make each of the individual segments light up.

In contrast, the common anode display is the opposite. In a common anode display, the positive terminal of the eight-shaped LEDs are connected together. They are then connected to pin 3 and pin 8. To turn on an individual segment, one of the pins is grounded. The diagram below shows the internal structure of the common anode seven-segment display. You turn on the individual segments to display a particular number, as shown below in the table.

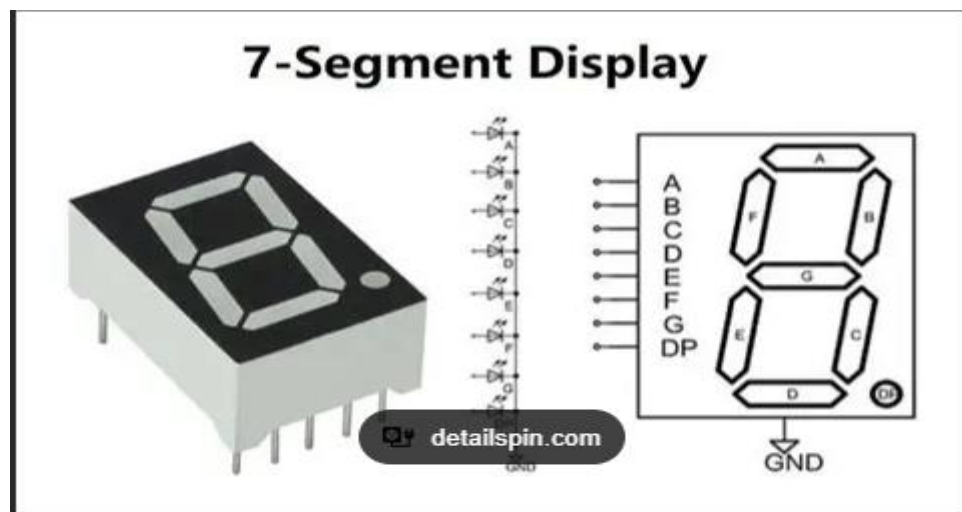


Fig: Seven Segment Display

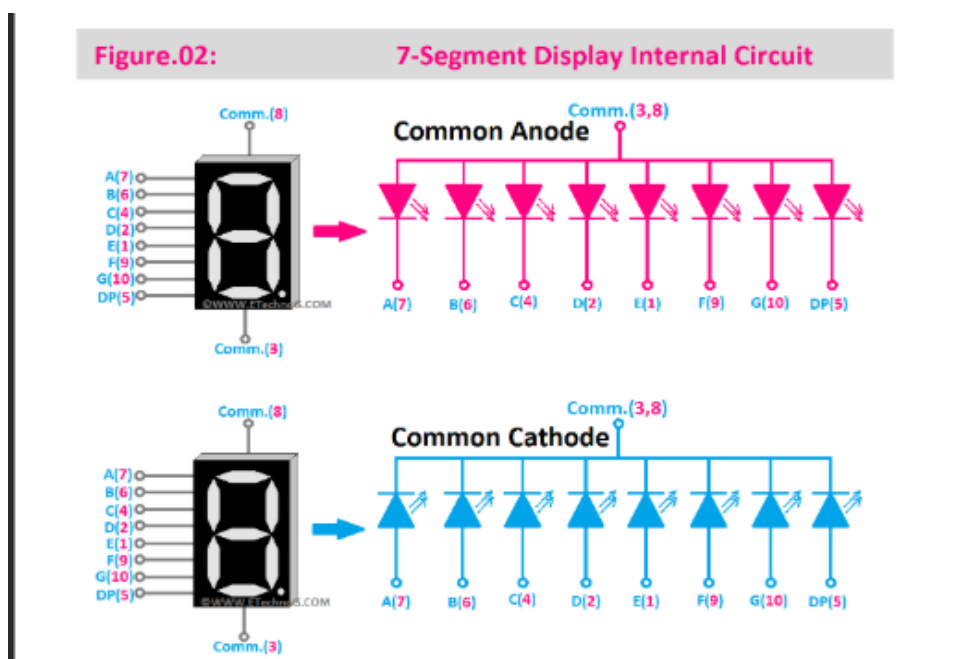


Fig. Types of Seven Segment Display

A	B	C	D	E	F	G	Display
1	1	1	1	1	1	0	0
0	1	1	0	0	0	0	1
1	1	0	1	1	0	1	2
1	1	1	1	0	0	1	3
0	1	1	0	0	1	1	4
1	0	1	1	0	1	1	5
1	0	1	1	1	1	1	6
1	1	1	0	0	0	0	7
1	1	1	1	1	1	1	8
1	1	1	1	0	1	1	9

Table: Individual Segments to Display a Particular Number

Source Code

```

#define segA 2//connecting segment A to PIN2

#define segB 3// connecting segment B to PIN3

#define segC 4// connecting segment C to PIN4

#define segD 5// connecting segment D to PIN5

#define segE 6// connecting segment E to PIN6

#define segF 7// connecting segment F to PIN7

#define segG 8// connecting segment G to PIN8


int COUNT=0;//count integer for 0-9 increment


void setup()
{
    for (int i=2;i<9;i++)
    {
        pinMode(i, OUTPUT);// taking all pins from 2-8 as output
    }
}

```

```
}
```

```
void loop()
```

```
{
```

```
    switch (COUNT)
```

```
    {
```

```
        case 0://when count value is zero show"0" on disp
```

```
        digitalWrite(segA, HIGH);
```

```
        digitalWrite(segB, HIGH);
```

```
        digitalWrite(segC, HIGH);
```

```
        digitalWrite(segD, HIGH);
```

```
        digitalWrite(segE, HIGH);
```

```
        digitalWrite(segF, HIGH);
```

```
        digitalWrite(segG, LOW);
```

```
        break;
```

```
        case 1:// when count value is 1 show"1" on disp
```

```
        digitalWrite(segA, LOW);
```

```
        digitalWrite(segB, HIGH);
```

```
        digitalWrite(segC, HIGH);
```

```
        digitalWrite(segD, LOW);
```

```
        digitalWrite(segE, LOW);
```

```
        digitalWrite(segF, LOW);
```

```
        digitalWrite(segG, LOW);
```

```
        break;
```

```
case 2:// when count value is 2 show"2" on disp
```

```
digitalWrite(segA, HIGH);
```

```
digitalWrite(segB, HIGH);
```

```
digitalWrite(segC, LOW);
```

```
digitalWrite(segD, HIGH);
```

```
digitalWrite(segE, HIGH);
```

```
digitalWrite(segF, LOW);
```

```
digitalWrite(segG, HIGH);
```

```
break;
```

```
case 3:// when count value is 3 show"3" on disp
```

```
digitalWrite(segA, HIGH);
```

```
digitalWrite(segB, HIGH);
```

```
digitalWrite(segC, HIGH);
```

```
digitalWrite(segD, HIGH);
```

```
digitalWrite(segE, LOW);
```

```
digitalWrite(segF, LOW);
```

```
digitalWrite(segG, HIGH);
```

```
break;
```

```
case 4:// when count value is 4 show"4" on disp
```

```
digitalWrite(segA, LOW);
```

```
digitalWrite(segB, HIGH);
```

```
digitalWrite(segC, HIGH);
```

```
digitalWrite(segD, LOW);
```

```
digitalWrite(segE, LOW);  
  
digitalWrite(segF, HIGH);  
  
digitalWrite(segG, HIGH);  
  
break;
```

```
case 5:// when count value is 5 show"5" on disp
```

```
digitalWrite(segA, HIGH);  
  
digitalWrite(segB, LOW);  
  
digitalWrite(segC, HIGH);  
  
digitalWrite(segD, HIGH);  
  
digitalWrite(segE, LOW);  
  
digitalWrite(segF, HIGH);  
  
digitalWrite(segG, HIGH);  
  
break;
```

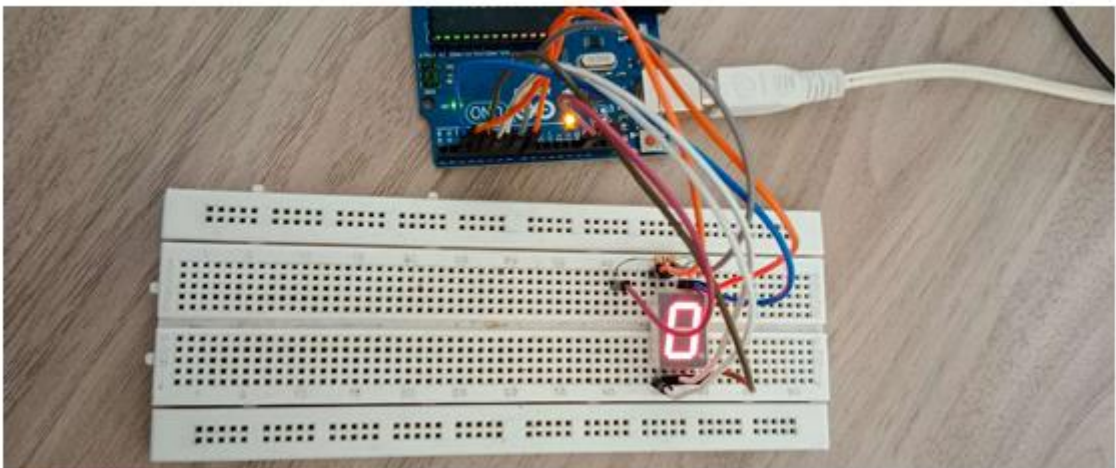
```
case 6:// when count value is 6 show"6" on disp
```

```
digitalWrite(segA, HIGH);  
  
digitalWrite(segB, LOW);  
  
digitalWrite(segC, HIGH);  
  
digitalWrite(segD, HIGH);  
  
digitalWrite(segE, HIGH);  
  
digitalWrite(segF, HIGH);  
  
digitalWrite(segG, HIGH);  
  
break;
```

```
case 7:// when count value is 7 show"7" on disp  
digitalWrite(segA, HIGH);  
digitalWrite(segB, HIGH);  
digitalWrite(segC, HIGH);  
digitalWrite(segD, LOW);  
digitalWrite(segE, LOW);  
digitalWrite(segF, LOW);  
digitalWrite(segG, LOW);  
break;  
  
case 8:// when count value is 8 show"8" on disp  
digitalWrite(segA, HIGH);  
digitalWrite(segB, HIGH);  
digitalWrite(segC, HIGH);  
digitalWrite(segD, HIGH);  
digitalWrite(segE, HIGH);  
digitalWrite(segF, HIGH);  
digitalWrite(segG, HIGH);  
break;  
  
case 9:// when count value is 9 show"9" on disp  
digitalWrite(segA, HIGH);  
digitalWrite(segB, HIGH);  
digitalWrite(segC, HIGH);  
digitalWrite(segD, HIGH);  
digitalWrite(segE, LOW);
```

```
    digitalWrite(segF, HIGH);  
    digitalWrite(segG, HIGH);  
    break;  
break;  
}  
  
if (COUNT<10)  
{  
    COUNT++;  
    delay(1000);///increment count integer for every second  
}  
if (COUNT==10)  
{  
    COUNT=0;/// if count integer value is equal to 10, reset it to zero.  
    delay(1000);  
}  
}
```

Output



Result:

Conclusion:

Post-Lab Questions

1. How to display a specific number continuously?
2. What changes for common anode display?
3. Why multiplexing is used in multi-digit display?
4. How to display alphabets?
5. What happens if resistor value is too small?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 07

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: To interface the DHT11 temperature sensor with Arduino and display temperature readings.

Objectives

- To understand working of DHT11 sensor
- To read digital sensor data using Arduino
- To display temperature values on serial monitor
- To learn single-wire communication protocol

Pre-Lab Questions – Interfacing of Temperature Sensor DHT11 with Arduino

1. What type of sensor is DHT11 and what parameters does it measure?
2. Why is a pull-up resistor required on the data pin?
3. Which communication protocol is used by the DHT11 sensor?
4. What is the temperature measurement range of DHT11?
5. Which Arduino library is commonly used to interface DHT11?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	Arduino Board	Arduino Board	1
2	Breadboard	-	1
3	Power Supply	+5V DC Regulated	1
4	DHT11 sensor	-	1
5	220 ohm resistor	220 Ohm resistor	1
6	Jumper Wires	-	10

Theory:

The DHT11 is a low-cost digital temperature and humidity sensor widely used in embedded and IoT applications. It contains a thermistor for measuring temperature and a capacitive humidity sensing element, along with an internal 8-bit microcontroller that converts the analog measurements into a

calibrated digital signal. Because the sensor performs internal signal conditioning and analog-to-digital conversion, the output is provided directly in digital form, eliminating the need for external ADC circuitry.

The sensor communicates with a microcontroller using a single-wire serial communication protocol. Only one data pin is required for data transfer, along with VCC and GND connections. A pull-up resistor (typically 10 k Ω) is connected between the data line and VCC to maintain stable logic levels. The microcontroller sends a start signal to the sensor, after which the DHT11 responds by transmitting a 40-bit data packet containing integral humidity, decimal humidity, integral temperature, decimal temperature and a checksum byte for error detection.

The DHT11 operates at 3.3V to 5V supply voltage and measures temperature in the range 0°C to 50°C with an accuracy of approximately $\pm 2^\circ\text{C}$, and humidity from 20% to 90% RH with $\pm 5\%$ accuracy. The sampling period is about 1 second, meaning readings should not be taken more frequently than once per second. Due to its simplicity, reliability and low power consumption, the DHT11 is commonly used in weather monitoring systems, home automation, air quality monitoring and environmental data logging applications.

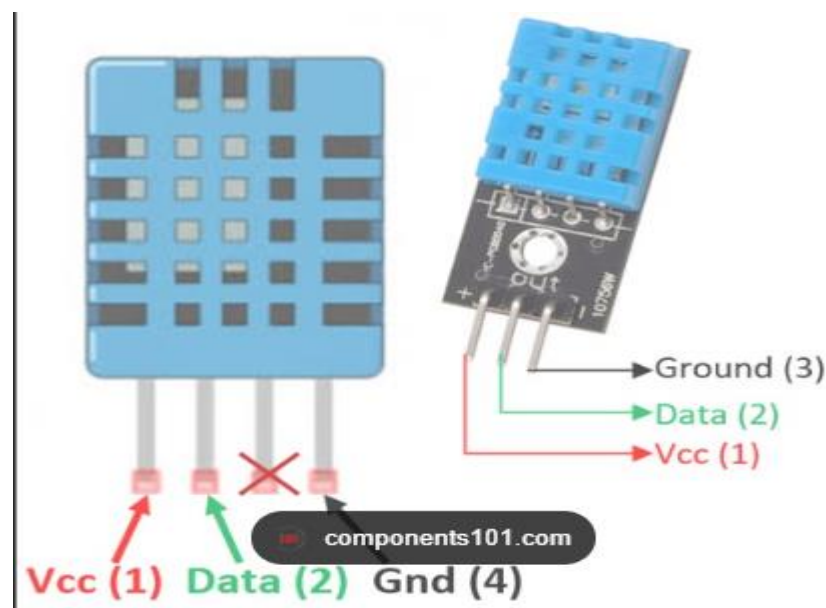


Fig. DHT 11 Temp Sensor Pin-Out

Source Code:

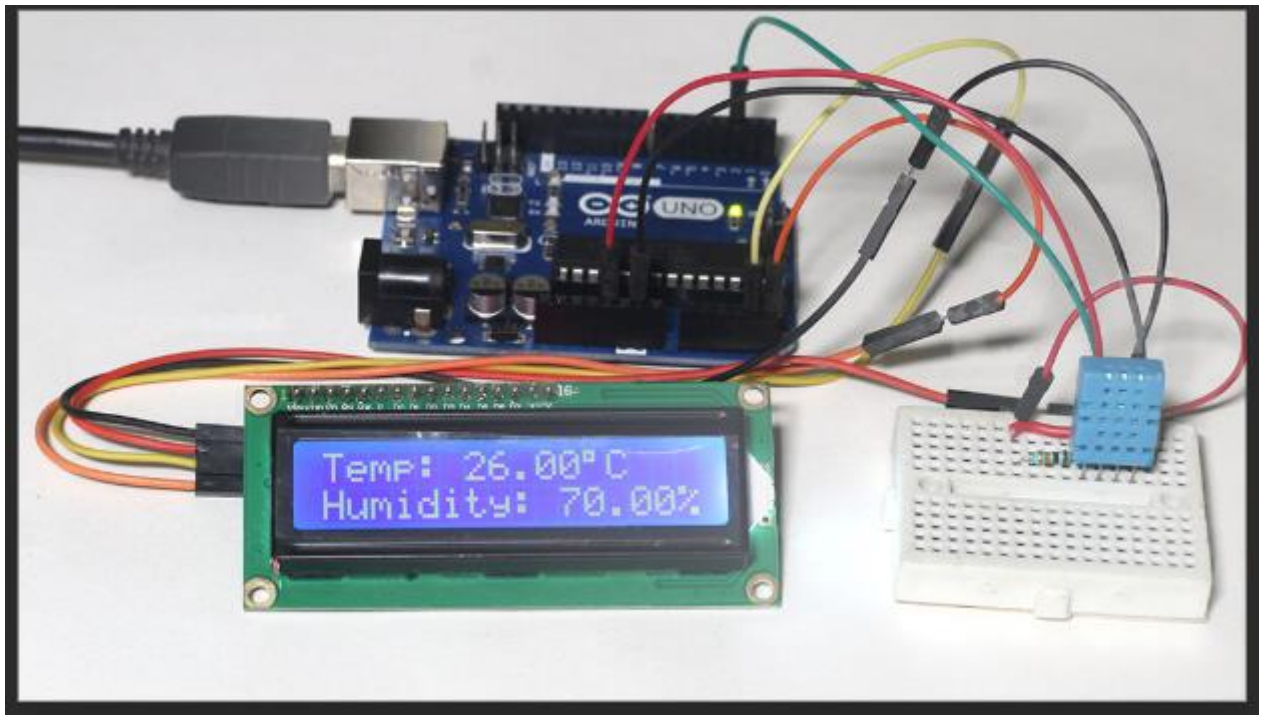
```
#include <DHT.h>

#define DHTPIN 2

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
```

```
void setup() {  
    Serial.begin(9600);  
    dht.begin();  
}  
  
void loop() {  
    float temp = dht.readTemperature();  
    Serial.print("Temperature: ");  
    Serial.print(temp);  
    Serial.println(" °C");  
    delay(2000);  
}
```



Result:

Conclusion:

Post-Lab Questions – Interfacing of Temperature Sensor DHT11 with Arduino

1. Why is a delay required between consecutive readings from DHT11?
2. What will happen if the pull-up resistor is not connected?
3. How can humidity data also be read from the same sensor?
4. How can the measured temperature be displayed on an LCD instead of serial monitor?
5. What is the difference between DHT11 and DHT22 sensors?



Laboratory Report

Experiment No :	
Batch Code:	
Date of Lab:	
Date of Submission:	
Experiment–Course Outcome (CO) Mapping	
CO 3: Write assembly language programs for microcontroller based systems using instruction set.	
Evaluation [15 Marks]	
Performance in Experiment [03]	
Virtual Lab [02]	
Journal Submission [05]	
Viva-Voce [05]	
Overall Marks [15]	
Sign of Course Coordinator	

EXPERIMENT NO: 08

	Shirpur Education Society's R. C. Patel Institute of Technology, Shirpur <i>An Autonomous Institute</i> Affiliated to DBATU, Lonere (M.S.), Approved By AICTE, New Delhi and Govt. of Maharashtra (DTE: EN - 5172), Accredited by NAAC 'A' Grade, Bangalore.		
Name of Student:			
Division:		Roll No:	
Performed Date:		Checked Date	

Aim: To interface a servo motor with Arduino and control its angular position.

Objectives

- To understand the working principle of servo motor
- To control servo position using PWM signal
- To generate different rotation angles using Arduino program
- To learn use of Servo library

Pre-Lab Questions – Interfacing of Servo Motor with Arduino

1. What is a servo motor and how is it different from a DC motor?
2. What type of signal is required to control the position of a servo motor?
3. What is PWM and why is it used in servo control?
4. What is the typical operating voltage and rotation range of a hobby servo motor?
5. Which Arduino library is used to control a servo motor?

Equipment's required:

Sr. No.	Name of Equipment	Specifications	Quantity
1	Arduino Board	Arduino Board	1
2	Breadboard	-	1
3	Power Supply	+5V DC Regulated	1
4	Servo Motor	5V	1
5	220 ohm resistor	220 Ohm resistor	1
6	Jumper Wires	-	10

Theory:

A servo motor is a precision rotary actuator that can rotate to a specific angular position based on control signals. It consists of a DC motor, gear mechanism, position feedback potentiometer and internal control circuit enclosed in one unit. The position of the shaft is controlled using a PWM (Pulse Width Modulation) signal where the width of the pulse determines the angle of rotation.

Typically, the servo expects a pulse every 20 ms: a pulse of about 1 ms rotates the shaft to 0°, 1.5 ms moves it to 90°, and 2 ms moves it to 180°. The internal feedback system continuously compares the commanded position with the actual position and adjusts the motor until the desired angle is reached, allowing accurate and stable positioning.



Fig. Pin Connections of Servo Motor

A servo motor typically has three pins, each serving a specific purpose for its operation. Here's a Breakdown of a common three-pin servo motor pin out:

1. Power Pin (Vcc): This pin is used to supply the voltage required to power the servo motor. For most standard hobby servos, the voltage range is typically between 4.8V to 6V. This pin is usually connected to a positive power supply.
2. Ground Pin (GND): This pin is the ground connection for the servo motor. It is connected to the ground of the power supply. Ensuring a common ground between the servo and the controlling device (such as a microcontroller) is essential for proper operation.
3. Control Pin (Signal): This pin receives the control signal from the controller (like an Arduino or another microcontroller). The control signal is a Pulse Width Modulation (PWM) signal, which determines the position of the servo motor. The duration of the pulse typically ranges from 1ms to 2ms, where 1ms usually corresponds to one extreme of the servo's range and 2ms to the other extreme. A 1.5ms pulse generally represents the neutral or centre position.

Source Code

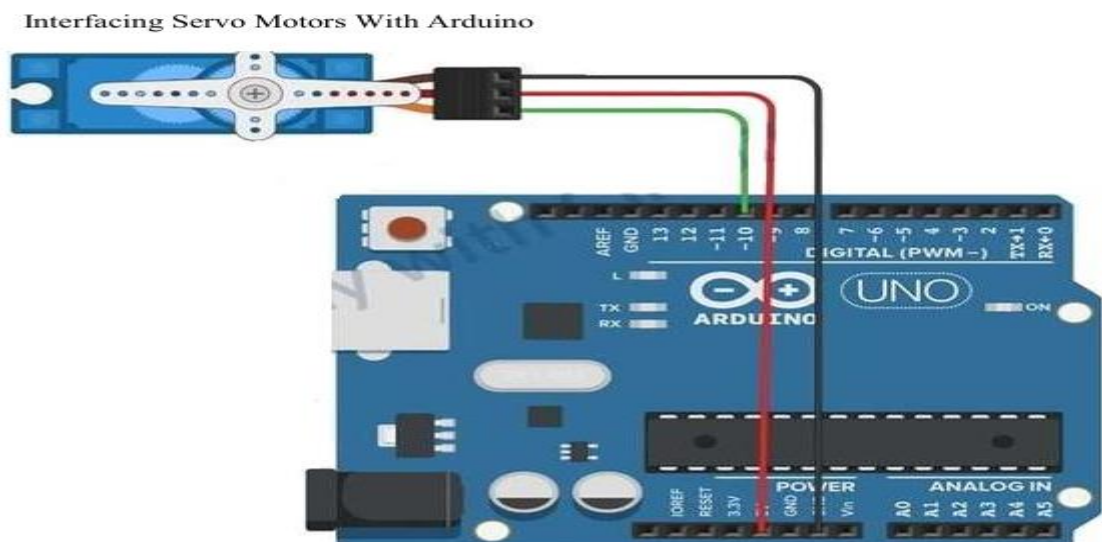
```
#include <Servo.h>

#define SERVO_PIN 10

Servo myservo;           // create servo object to control a servo

int pos = 0;             // variable to store the servo position
```

```
void setup() {  
  
  myservo.attach(SERVO_PIN);      // attaches the servo on pin 9 to the servo object  
  
  myservo.write(pos);              // go to position zero first  
  
  delay(2000);                     // wait for some time  
  
}  
  
void loop() {  
  
  delay(1000);                     // delay of 1 second before start  
  
  for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees in steps of 1 degree  
  
    myservo.write(pos);            // tell servo to go to position in variable 'pos'  
  
    delay(15);                     // waits 15 ms for the servo to reach the position  
  
  }  
  
  delay(1000);                     // delay of 1 second before changing direction of rotation  
  
  for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees  
  
    myservo.write(pos);            // tell servo to go to position in variable 'pos'  
  
    delay(15);                     // waits 15 ms for the servo to reach the position  
  
  }  
  
}
```



Result:

Conclusion:

Post-Lab Questions – Interfacing of Servo Motor with Arduino

1. How does changing the pulse width affect the angle of rotation?
2. What happens if the servo motor is powered with insufficient voltage?
3. Why is a common ground connection necessary between Arduino and servo?
4. How can you rotate the servo to a specific angle and hold it there?
5. Why is an external power supply recommended for larger servo motors?